Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра вычислительных систем

ОТЧЕТ
по практической работе 7

по дисциплине «**Основы разработки на Java**»

Выполнил:
студент гр. ИС-542
«11» октября 2026 г. _____ /Кузугашев С.В./

Проверил:
Ассистент кафедры ВС
«12» октября 2026 г. _____ /Гузенков С.К./

Оценка «_____»

Новосибирск 2026

Оглавление

Задание 3
Выполнение работы 4
Краткая теория 4
Структура проекта 6
Задача 1. Каталог курсов в UTF-8 6
Задача 2. Buffered Reader/Writer и граница bytes → chars 7
Задача 3. Try-with-resources и порядок закрытия 8
Задача 4. Ленивые строки и владение Stream 9
Задача 5. Бинарный формат каталога 10
Задача 6. Временный файл и atomic move 10
Задача 7. Частично повреждённый импорт 11
Задача 8. Потоковый импорт большого файла 12
Задача 9. Версионируемый формат и миграция 13
Ошибки и их исправление 15
Заключение 16
Приложение 17

Задание

Дисциплина «Программирование». Лабораторная работа №7. Тема: работа с файлами и потоками ввода-вывода Java — кодировки, буферизация, try-with-resources, ленивые Stream, бинарные и версионируемые форматы, атомарная замена и потоковый импорт. Среда: OpenJDK 21.

Исходный код лабораторной работы опубликован в открытом репозитории: https://github.com/Forlein/SibGUTI-Java-Labs (папка WORK_Java/laboratory_work_7).

В работе реализованы 9 заданий:

1. Каталог курсов: сохранение и чтение в UTF-8, обработка отсутствующего файла и каталога вместо файла.

2. Текстовая обработка через BufferedReader/BufferedWriter со счётчиками строк и символов, объяснение границы bytes → chars.

3. Try-with-resources: pipeline с исключением в середине и доказательство правильного порядка close.

4. Files.lines только в try-with-resources; воспроизведение ошибочного возврата Stream наружу.

5. Бинарный формат: magic, version, count и записи через DataOutputStream/DataInputStream; отказ для неизвестной версии и обрезанного файла.

6. Временный файл в том же каталоге, flush/close и замена целевого файла через move с fallback без atomic move.

7. Импорт с ошибками и номерами строк: all-or-nothing, контракт результата зафиксирован отдельным типом.

8. Потоковый импорт большого файла: счётчики, ограниченный список ошибок, отмена после заданного порога.

9. Версионируемый формат: schema version, checksum, миграция v1 → v2, отказ будущей версии.

Выполнение работы

Ниже по каждой задаче приведён ключевой фрагмент кода (микро-листинг) и вывод программы; полные исходники лежат в папках 1–9 проекта.

Краткая теория

Файл в Java — это последовательность байтов. Класс Files выполняет операции над файловой системой, а Path задаёт путь. Текстовая кодировка определяет, как байты превращаются в символы. UTF-8 кодирует один Unicode code point одним–четырьмя байтами: русская буква занимает два байта, эмодзи — четыре. Если открыть UTF-8 файл в другой кодировке, получится «кракозябра», поэтому кодировку лучше указывать явно.

Граница bytes → chars проходит в InputStreamReader: он читает байты и декодирует их в UTF-16 char. Обратная граница — OutputStreamWriter: он кодирует char в байты. BufferedReader и BufferedWriter только буферизуют данные и уменьшают число обращений к диску. Один char — это UTF-16 code unit; эмодзи занимает две char, поэтому число char и число code points может различаться.

try-with-resources гарантированно вызывает close() у ресурсов, реализующих AutoCloseable. Ресурсы закрываются в порядке, обратном объявлению. Если в теле возникает исключение, close() всё равно выполняется; если close() тоже бросает исключение, оно добавляется как suppressed. Поэтому утечка файлового дескриптора при таком синтаксисе практически исключена.

Files.lines возвращает ленивый Stream, который держит открытый файловый дескриптор. Terminal operation (count, forEach, collect) не закрывает Stream. Владелец ресурса — тот, кто вызвал Files.lines; если Stream возвращают из метода, закрывать его обязан вызывающий. Безопаснее потреблять Stream внутри try-with-resources, тогда close() вызовется автоматически.

Бинарный формат хранит данные в удобном для программы виде. Magic header — сигнатура формата, version — номер схемы, count — количество записей. DataOutputStream пишет примитивы в big-endian, DataInputStream читает их в том же порядке. EOFException означает, что файл короче ожидаемого; неизвестную версию нужно отклонять до разбора записей, иначе можно неверно интерпретировать данные.

Публикация файла — это замена старого содержимого новым. Чтобы не оставить полузаписанный файл, новый каталог пишут во временный файл в том же каталоге, делают flush/close, а затем перемещают его на место целевого. Files.move с ATOMIC_MOVE выполняет замену одной операцией, если файловая система это поддерживает. Если atomic move недоступен, остаётся обычный move с REPLACE_EXISTING, но атомарность уже не гарантируется.

Импорт «всё или ничего» означает, что при любой ошибке каталог не публикуется частично. Ошибки собирают с номерами строк, а результат возвращают типом, который явно различает успех и отказ: например, record с полями courses и errors. Такой контракт не позволяет случайно использовать половину данных и делает поведение импортёра предсказуемым.

Большой файл нельзя читать через readAllLines: этот метод загружает все строки в память. Потоковый импорт читает по одной строке через BufferedReader, хранит только счётчики и ограниченный список ошибок. После заданного порога ошибок импорт можно отменить, чтобы не тратить время и память. Сложность по памяти остаётся O(1) относительно числа строк.

Версионируемый формат добавляет schema version и checksum. Версия говорит, как разбирать payload; checksum (например, CRC32) позволяет обнаружить повреждение до создания объектов. Миграция v1 → v2 — это преобразование старых записей в новую схему, например вычисление credits из hours. Будущую версию нельзя читать как старую: сначала проверяют версию, потом разбирают данные.

Главный принцип: ресурс должен закрывать тот, кто его открыл; ошибку нужно обрабатывать там, где можно принять решение, а не глотать; формат

должен явно хранить версию и контрольную сумму; публикация должна быть атомарной или явно all-or-nothing. Эти правила превращают учебный код в надёжный промышленный.

Структура проекта

Каждое задание вынесено в отдельную папку с номером, внутри которой лежит файл Main.java. Классы с одинаковыми именами (Course, Main и т. п.) продублированы в папках, поэтому задания собираются и запускаются по отдельности. Сборка: javac -encoding UTF-8 -d out <папка>\Main.java; запуск: java -Dstdout.encoding=UTF-8 -cp out Main.

Папка	Файлы и задание
1	Main.java — каталог курсов в UTF-8
2	Main.java — BufferedReader/BufferedWriter
3	Main.java — try-with-resources
4	Main.java — Files.lines и владение Stream
5	Main.java — бинарный формат
6	Main.java — временный файл и atomic move
7	Main.java — частично повреждённый импорт
8	Main.java — потоковый импорт большого файла
9	Main.java — версионируемый формат
Корень	README.md, Отчет_ЛР7_Java.docx

Задача 1. Каталог курсов в UTF-8

Задача показывает разницу между байтами файла и текстом программы. Files.write и Files.readAllLines работают с UTF-8 явно, поэтому русские названия курсов не зависят от кодировки системы. Проверка Files.isDirectory отделяет ошибку «вместо файла каталог» от нормальной ситуации «файла ещё нет»: отсутствующий файл можно создать, а каталог нельзя молча перезаписать.

Листинг – 1 Проверки и чтение каталога (1/Main.java)

```
1     if (Files.isDirectory(path)) {
2         throw new IOException("по пути находится каталог, а не файл");
3     }
4     if (!Files.exists(path)) {
5         Files.write(path, courses, StandardCharsets.UTF_8);
6         System.out.println("Файла не было — каталог создан");
```

```
7     }
8     for (String line : Files.readAllLines(path, StandardCharsets.UTF_8))
      {
9         System.out.println(line);
10    }
```

Вывод программы (1/Main.java)

```
1     Файла не было — каталог создан
2     Java;72
3     Базы данных;64
4     Сети;96
```

Файла сначала не было, поэтому программа создала его и записала три строки. При повторном запуске файл уже существует, и чтение возвращает те же строки. Если вместо файла создать каталог courses.txt, проверка Files.isDirectory выбросит IOException, и программа завершится с кодом 1.

Задача 2. Buffered Reader/Writer и граница bytes → chars

BufferedReader и BufferedWriter уменьшают число обращений к диску, но не меняют природу данных. Байты в char декодирует InputStreamReader, а char в байты кодирует OutputStreamWriter. Счётчик line.length() считает UTF-16 char, а codePointCount — реальные Unicode-символы. Поэтому эмодзи увеличивает счётчик char на две единицы, а code points — на одну.

Листинг – 2 Буферизованное чтение, запись и счётчики (2/Main.java)

```
1     try (BufferedReader reader = new BufferedReader(
2             new InputStreamReader(Files.newInputStream(input),
      StandardCharsets.UTF_8));
3          BufferedWriter writer = new BufferedWriter(
4                 new OutputStreamWriter(Files.newOutputStream(output),
      StandardCharsets.UTF_8))) {
5         String line;
6         while ((line = reader.readLine()) != null) {
7             lines++;
8             chars += line.length();
9             codePoints += line.codePointCount(0, line.length());
10            writer.write(line);
11            writer.newLine();
12        }
13    }
```

Вывод программы (2/Main.java)

```
1     Создан демонстрационный файл: input.txt
2     Байты читает InputStream, символы — InputStreamReader; обратно —
      OutputStreamWriter.
```

В файле две строки. UTF-16 char — 13, code points — 12: эмодзи 😀 занимает суррогатную пару. InputStreamReader — граница bytes → chars, OutputStreamWriter — обратная граница chars → bytes.

Задача 3. Try-with-resources и порядок закрытия

try-with-resources — это не просто сокращённая запись try/finally. Он гарантирует вызов close() даже при исключении в теле блока и закрывает ресурсы в порядке, обратном объявлению. В задаче исключение возникает после чтения и записи, но оба ресурса всё равно попадают в список CLOSED. По этому списку видно, что writer закрылся раньше reader.

Листинг – 3 Pipeline с исключением и трассировкой close (3/Main.java)

```
1     static class Resource implements Closeable {
2         final String name;
3
4         Resource(String name) { this.name = name; }
5         String read() { return "данные"; }
6         void write(String text) { System.out.println("Записано: " +
      text); }
7
8         @Override
9         public void close() { CLOSED.add(name); }
10    }
11
12    try (Resource reader = new Resource("reader");
13         Resource writer = new Resource("writer")) {
14        writer.write(reader.read());
15        throw new IOException("сбой в середине pipeline");
16    }
```

Вывод программы (3/Main.java)

```
1     Записано: данные
2     Исключение: сбой в середине pipeline
3     Порядок close: [writer, reader]
```

Исключение возникло после записи, но writer и reader всё равно закрыты. Порядок [writer, reader] обратен объявлению: writer объявлен вторым, поэтому закрывается первым.

Задача 4. Ленивые строки и владение Stream

Files.lines не читает файл сразу: он создаёт ленивый Stream, который держит файловый дескриптор. Terminal operation count() только запускает чтение, но не вызывает close(). Если вернуть Stream из метода, владельцем ресурса становится вызывающий код. Ошибка в том, что вызывающий может воспринимать Stream как обычную коллекцию и забыть закрыть его.

Листинг – 4 Ошибочный возврат Stream и корректный try-with-resources (4/Main.java)

```
1     static Stream<String> broken(Path path) throws IOException {
2         return Files.lines(path, StandardCharsets.UTF_8);
3     }
4
5     Stream<String> stream = broken(path);
6     System.out.println("Строк: " + stream.count());
7     System.out.println("Stream не закрыт: владелец — вызывающий код");
8     stream.close();
9
10    try (Stream<String> lines = Files.lines(path,
      StandardCharsets.UTF_8)) {
11        System.out.println("Правильно, строк: " + lines.count());
12    }
```

Вывод программы (4/Main.java)

```
1     Строк: 2
2     Stream не закрыт: владелец — вызывающий код
3     Правильно, строк: 2
```

count() прочитал две строки, но не закрыл Stream: файловый дескриптор остаётся открытым. Владелец ресурса — вызывающий код, поэтому после возврата Stream его нужно закрыть. Корректный вариант с try-with-resources делает это автоматически.

Задача 5. Бинарный формат каталога

Бинарный файл не содержит текстовых разделителей, поэтому программа сама задаёт порядок полей. Magic header отделяет формат от случайных файлов, version задаёт схему, count говорит, сколько записей читать. DataOutputStream и DataInputStream работают с примитивами в big-endian. Неизвестную версию и нехватку байтов нужно проверять до использования данных.

Листинг – 5 Заголовок и записи бинарного формата (5/Main.java)

```
1     try (DataOutputStream out = new
      DataOutputStream(Files.newOutputStream(path))) {
2         out.writeInt(MAGIC);
3         out.writeInt(VERSION);
4         out.writeInt(courses.size());
5         for (Course course : courses) {
6             out.writeUTF(course.name());
7             out.writeInt(course.hours());
8         }
9     }
10
11    if (in.readInt() != MAGIC) throw new IOException("неверный magic
      header");
12    int version = in.readInt();
13    if (version != VERSION) throw new IOException("неизвестная версия: "
      + version);
14    int count = in.readInt();
15    for (int i = 0; i < count; i++) {
16        System.out.println(in.readUTF() + " — " + in.readInt());
17    }
```

Вывод программы (5/Main.java)

```
1     Java — 72
2     Сети — 96
```

Файл содержит magic, version=1, count=2 и две записи. Чтение идёт в том же порядке, что и запись. Если изменить version на 99, программа выбросит IOException до чтения записей; если обрезать файл, сработает EOFException.

Задача 6. Временный файл и atomic move

Запись прямо в целевой файл опасна: при сбое останется половина каталога. Поэтому новый каталог пишется во временный файл в том же

каталоге, затем flush/close гарантируют, что данные ушли на диск. Files.move с ATOMIC_MOVE заменяет целевой файл одной операцией. Если файловая система не поддерживает atomic move, программа делает обычный move и предупреждает, что атомарность не гарантирована.

Листинг – 6 Временный файл и замена целевого файла (6/Main.java)

```
1     temp = Files.createTempFile(target.toAbsolutePath().getParent(),
      "courses", ".tmp");
2     try (BufferedWriter writer = Files.newBufferedWriter(temp,
      StandardCharsets.UTF_8)) {
3         writer.write("Java\nБазы данных\n");
4         writer.flush();
5     }
6
7     try {
8         Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING,
9                 StandardCopyOption.ATOMIC_MOVE);
10    } catch (AtomicMoveNotSupportedException e) {
11        Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING);
12    }
```

Вывод программы (6/Main.java)

```
1     Java
2     Базы данных
```

Новый каталог записан во временный файл в том же каталоге, затем заменён через ATOMIC_MOVE. На NTFS atomic move поддерживается; при AtomicMoveNotSupportedException выполняется обычная замена с предупреждением.

Задача 7. Частично повреждённый импорт

Импорт all-or-nothing не публикует частично разобранные данные. Ошибки собираются с номерами строк, а результат возвращается отдельным типом Result: в нём либо список courses, либо список errors. Если errors не пуст, courses принудительно пуст, поэтому вызывающий код не может случайно опубликовать половину каталога.

Листинг – 7 Контракт результата импорта (7/Main.java)

```
1     record Result(List<String> courses, List<String> errors) {
```

```
    }

    if (parts.length != 2 || parts[0].isBlank()) {
        errors.add("строка " + (i + 1) + ": нужно имя;часы");
        continue;
    }
    try {
        int hours = Integer.parseInt(parts[1]);
        courses.add(parts[0] + " — " + hours);
    } catch (NumberFormatException e) {
        errors.add("строка " + (i + 1) + ": неверные часы");
    }

    if (!errors.isEmpty()) {
        return new Result(List.of(), errors);
    }
    return new Result(courses, List.of());
```

Вывод программы (7/Main.java)

```
Импорт отклонён целиком:
строка 2: неверные часы
строка 3: нужно имя;часы
```

В файле три строки: корректная, с неверными часами и с пустым именем. Result вернул errors, а courses пуст, поэтому каталог не опубликован частично. Ошибки содержат номера строк 2 и 3.

Задача 8. Потоковый импорт большого файла

readAllLines загружает весь файл в память, поэтому для большого файла он не подходит. Потоковый импорт читает по одной строке через BufferedReader и хранит только счётчики и ограниченный список ошибок. После заданного порога ошибок цикл прерывается, поэтому память не растёт вместе с размером файла, а импорт можно остановить досрочно.

Листинг – 8 Построчный импорт с ограниченными ошибками (8/Main.java)

```
try (BufferedReader reader = Files.newBufferedReader(path,
StandardCharsets.UTF_8)) {
    String line;
    while ((line = reader.readLine()) != null) {
        lines++;
```

```
            String[] parts = line.split(";");
            try {
                Integer.parseInt(parts[1]);
                good++;
            } catch (RuntimeException e) {
                errors++;
                if (firstErrors.size() < 5) {
                    firstErrors.add("строка " + lines);
                }
            }
            if (errors >= 10) {
                System.out.println("Отмена после 10 ошибок");
                break;
            }
        }
    }
```

Вывод программы (8/Main.java)

```
Отмена после 10 ошибок
Строк: 1000, корректных: 990, ошибок: 10
Первые ошибки: [строка 100, строка 200, строка 300, строка 400,
строка 500]
```

Файл содержит 1000 строк, каждая сотая — ошибка. После 10-й ошибки импорт остановился на строке 1000: корректных 990, ошибок 10. В списке сохранены только первые 5 ошибок.

Задача 9. Версионируемый формат и миграция

Версия формата нужна, чтобы старые и новые файлы не смешивались. В версии 1 записи содержат только name и hours, в версии 2 добавляется credits. При чтении v1 программа мигрирует данные, вычисляя credits из hours. CRC32 считается по payload и проверяется до разбора объектов, поэтому повреждение не приводит к публикации неверных записей.

Листинг – 9 Версия, checksum и миграция v1 → v2 (9/Main.java)

```
int version = in.readInt();
if (version != 1 && version != 2) {
    throw new IOException("неизвестная версия: " + version);
}
int length = in.readInt();
int checksum = in.readInt();
byte[] data = in.readNBytes(length);
```

```
8     CRC32 crc = new CRC32();
9     crc.update(data);
10    if ((int) crc.getValue() != checksum) {
11        throw new IOException("checksum не совпал");
12    }
13
14    int credits = version == 1 ? Math.max(1, hours / 36) :
      payload.readInt();
```

Вывод программы (9/Main.java)

```
1     v1 -> v2: [Course[name=Java, hours=72, credits=2]]
2     v2: [Course[name=Java, hours=72, credits=2]]
3     Повреждение: checksum не совпал
```

v1 не содержит credits, поэтому при чтении credits вычисляется как hours / 36. v2 читается напрямую. Повреждение последнего байта обнаружено по CRC32 до разбора объектов, поэтому неверные данные не были опубликованы.

Ошибки и их исправление

Отсутствие файла и каталог вместо файла (задача 1). Files.exists() и Files.isDirectory() проверяются до чтения и записи; вместо молчаливого пропуска выбрасывается IOException, и программа завершается с ненулевым кодом.

Кодировка UTF-8 (задачи 1–2). Кодировка платформы может отличаться, поэтому во всех текстовых операциях явно указан StandardCharsets.UTF_8, а сборка и запуск выполняются с -encoding UTF-8 и -Dstdout.encoding=UTF-8.

Порядок закрытия ресурсов (задача 3). try-with-resources закрывает ресурсы в обратном порядке объявления; для pipeline это writer, затем reader. Ручной close в неправильном порядке мог бы потерять буферизованные данные.

Утечка файлового дескриптора (задача 4). Files.lines возвращает ленивый Stream, владеющий дескриптором. Возврат наружу без закрытия оставляет ресурс открытым; корректно потреблять поток внутри try-with-resources.

Неизвестная версия и обрезанный файл (задачи 5, 9). Версия проверяется до разбора записей, а EOF и несовпадение CRC32 преобразуются в понятные исключения; частично прочитанные объекты не публикуются.

Атомарная замена и частичный импорт (задачи 6–7). Временный файл создаётся в том же каталоге, чтобы move был атомарным; при отсутствии ATOMIC_MOVE выполняется fallback с предупреждением. Импорт all-or-nothing не публикует половину каталога.

Заключение

В ходе выполнения лабораторной работы №7 были изучены и применены на практике средства файлового ввода-вывода Java (OpenJDK 21): кодировки, буферизация, try-with-resources, ленивые Stream, бинарные и версионируемые форматы, атомарная замена и потоковая обработка больших файлов.

В задачах 1–3 отработаны текстовая обработка и управление ресурсами: явная граница bytes → chars через InputStreamReader/OutputStreamWriter, счётчики строк и символов, а также гарантированное закрытие reader и writer в обратном порядке.

В задачах 4–6 исследованы ленивые потоки и надёжная публикация файлов: доказано, что Files.lines требует try-with-resources, реализованы бинарный формат с magic/version/count и замена целевого файла через временный файл и atomic move.

В задачах 7–9 построены контракты импорта и версионирования: all-or-nothing результат с номерами строк, потоковый импорт с ограниченной памятью и отменой по порогу, schema version с CRC32 и миграцией v1 → v2.

Все 9 заданий размещены в отдельных папках, каждое задание компилируется и выполняется без ошибок. Работа выполнена в полном объёме.

Приложение

Листинг – 10 1/Main.java

```
1    /*
2    Задание 1. Сохраните каталог курсов в UTF-8, прочитайте, обработайте
     отсутствие файла и существующий каталог вместо файла. Не ловите
     ошибку только ради печати и продолжения с ложным успехом.
3    */
4    import java.io.IOException;
5    import java.nio.charset.StandardCharsets;
6    import java.nio.file.Files;
7    import java.nio.file.Path;
8    import java.nio.file.Paths;
9    import java.util.List;
10
11   public class Main {
12       public static void main(String[] args) {
13           Path path = Paths.get("courses.txt");
14           List<String> courses = List.of("Java;72", "Базы данных;64",
     "Сети;96");
15
16           try {
17               if (Files.isDirectory(path)) {
18                   throw new IOException("по пути находится каталог, а
     не файл");
19               }
20               if (!Files.exists(path)) {
21                   Files.write(path, courses, StandardCharsets.UTF_8);
22                   System.out.println("Файла не было — каталог
     создан");
23               }
24               for (String line : Files.readAllLines(path,
     StandardCharsets.UTF_8)) {
25                   System.out.println(line);
26               }
27           } catch (IOException e) {
28               System.err.println("Ошибка: " + e.getMessage());
29               System.exit(1);
30           }
31       }
32   }
```

Листинг – 11 2/Main.java

```
1    /*
2    Задание 2. `Reader`/`Writer`
3
```

```
4     Перепишите текстовую обработку через buffered reader/writer,
      считайте строки и символы. Объясните границу bytes → chars.
5     */
6     import java.io.BufferedReader;
7     import java.io.BufferedWriter;
8     import java.io.IOException;
9     import java.io.InputStreamReader;
10    import java.io.OutputStreamWriter;
11    import java.nio.charset.StandardCharsets;
12    import java.nio.file.Files;
13    import java.nio.file.Path;
14    import java.nio.file.Paths;
15
16    public class Main {
17        public static void main(String[] args) {
18            Path input = Paths.get("input.txt");
19            Path output = Paths.get("output.txt");
20
21            try {
22                Files.writeString(input, "Привет\nJava 😀\n",
      StandardCharsets.UTF_8);
23                int lines = 0;
24                int chars = 0;
25                int codePoints = 0;
26
27                try (BufferedReader reader = new BufferedReader(
28                        new
      InputStreamReader(Files.newInputStream(input),
      StandardCharsets.UTF_8));
29                     BufferedWriter writer = new BufferedWriter(
30                             new
      OutputStreamWriter(Files.newOutputStream(output),
      StandardCharsets.UTF_8))) {
31                    String line;
32                    while ((line = reader.readLine()) != null) {
33                        lines++;
34                        chars += line.length();
35                        codePoints += line.codePointCount(0,
      line.length());
36                        writer.write(line);
37                        writer.newLine();
38                    }
39                }
40
41                System.out.println("Строк: " + lines + ", char: " +
      chars + ", code points: " + codePoints);
42                System.out.println("Байты читает InputStream, символы —
      InputStreamReader; обратно — OutputStreamWriter.");
```

```
43            } catch (IOException e) {
44                System.err.println("Ошибка: " + e.getMessage());
45                System.exit(1);
46            }
47        }
48    }
```

Листинг – 12 3/Main.java

```
1     /*
2     Задание 3. Try-with-resources
3
4     Создайте pipeline чтения и записи, в середине которого возникает
      исключение. Докажите закрытие обоих ресурсов и правильный порядок
      close.
5     */
6     import java.io.Closeable;
7     import java.io.IOException;
8     import java.util.ArrayList;
9     import java.util.List;
10
11    public class Main {
12        static final List<String> CLOSED = new ArrayList<>();
13
14        static class Resource implements Closeable {
15            final String name;
16
17            Resource(String name) {
18                this.name = name;
19            }
20
21            String read() {
22                return "данные";
23            }
24
25            void write(String text) {
26                System.out.println("Записано: " + text);
27            }
28
29            @Override
30            public void close() {
31                CLOSED.add(name);
32            }
33        }
34
35        public static void main(String[] args) {
```

```
36            try (Resource reader = new Resource("reader");
37                 Resource writer = new Resource("writer")) {
38                writer.write(reader.read());
39                throw new IOException("сбой в середине pipeline");
40            } catch (IOException e) {
41                System.out.println("Исключение: " + e.getMessage());
42            }
43            System.out.println("Порядок close: " + CLOSED);
44        }
45    }
```

Листинг – 13 4/Main.java

```
1     /*
2     Задание 4. Ленивые строки
3
4     Используйте `Files.lines` только внутри try-with-resources.
      Воспроизведите ошибочный возврат stream наружу и объясните, кто
      обязан владеть ресурсом.
5     */
6     import java.io.IOException;
7     import java.nio.charset.StandardCharsets;
8     import java.nio.file.Files;
9     import java.nio.file.Path;
10    import java.util.stream.Stream;
11
12    public class Main {
13        static Stream<String> broken(Path path) throws IOException {
14            return Files.lines(path, StandardCharsets.UTF_8);
15        }
16
17        public static void main(String[] args) throws IOException {
18            Path path = Files.createTempFile("lines", ".txt");
19            Files.writeString(path, "один\ндва\n",
      StandardCharsets.UTF_8);
20
21            Stream<String> stream = broken(path);
22            System.out.println("Строк: " + stream.count());
23            System.out.println("Stream не закрыт: владелец — вызывающий
      код");
24            stream.close();
25
26            try (Stream<String> lines = Files.lines(path,
      StandardCharsets.UTF_8)) {
27                System.out.println("Правильно, строк: " +
      lines.count());
28            }
```

```
29
30            Files.deleteIfExists(path);
31        }
32    }
```

Листинг – 14 5/Main.java

```
1     /*
2     Задание 5. Бинарный формат
3
4     Запишите magic header, version, количество и записи через
      `DataOutputStream`; прочитайте через `DataInputStream`. Отклоняйте
      неизвестную версию и обрезанный файл.
5     */
6     import java.io.DataInputStream;
7     import java.io.DataOutputStream;
8     import java.io.EOFException;
9     import java.io.IOException;
10    import java.nio.file.Files;
11    import java.nio.file.Path;
12    import java.nio.file.Paths;
13    import java.util.List;
14
15    public class Main {
16        static final int MAGIC = 0x12345678;
17        static final int VERSION = 1;
18
19        record Course(String name, int hours) {
20        }
21
22        public static void main(String[] args) {
23            Path path = Paths.get("courses.bin");
24            List<Course> courses = List.of(new Course("Java", 72), new
      Course("Сети", 96));
25
26            try {
27                try (DataOutputStream out = new
      DataOutputStream(Files.newOutputStream(path))) {
28                    out.writeInt(MAGIC);
29                    out.writeInt(VERSION);
30                    out.writeInt(courses.size());
31                    for (Course course : courses) {
32                        out.writeUTF(course.name());
33                        out.writeInt(course.hours());
34                    }
35                }
36
```

```
37              try (DataInputStream in = new
    DataInputStream(Files.newInputStream(path))) {
38                  if (in.readInt() != MAGIC) {
39                      throw new IOException("неверный magic header");
40                  }
41                  int version = in.readInt();
42                  if (version != VERSION) {
43                      throw new IOException("неизвестная версия: " +
    version);
44                  }
45                  int count = in.readInt();
46                  for (int i = 0; i < count; i++) {
47                      System.out.println(in.readUTF() + " — " +
    in.readInt());
48                  }
49              }
50          } catch (EOFException e) {
51              System.err.println("Файл обрезан");
52              System.exit(1);
53          } catch (IOException e) {
54              System.err.println("Ошибка: " + e.getMessage());
55              System.exit(1);
56          }
57      }
58  }
```

Листинг – 15 6/Main.java

```
1   /*
2   Задание 6. Временный файл и move
3
4   Записывайте новый каталог во временный файл того же каталога,
    flush/close, затем заменяйте целевой файл move-операцией.
    Обработайте отсутствие atomic move на файловой системе.
5   */
6   import java.io.BufferedWriter;
7   import java.io.IOException;
8   import java.nio.charset.StandardCharsets;
9   import java.nio.file.AtomicMoveNotSupportedException;
10  import java.nio.file.Files;
11  import java.nio.file.Path;
12  import java.nio.file.Paths;
13  import java.nio.file.StandardCopyOption;
14
15  public class Main {
16      public static void main(String[] args) {
17          Path target = Paths.get("courses.txt");
18          Path temp = null;
```

```
19
20          try {
21              temp =
    Files.createTempFile(target.toAbsolutePath().getParent(), "courses",
    ".tmp");
22              try (BufferedWriter writer =
    Files.newBufferedWriter(temp, StandardCharsets.UTF_8)) {
23                  writer.write("Java\nБазы данных\n");
24                  writer.flush();
25              }
26
27              try {
28                  Files.move(temp, target,
29                          StandardCopyOption.REPLACE_EXISTING,
30                          StandardCopyOption.ATOMIC_MOVE);
31              } catch (AtomicMoveNotSupportedException e) {
32                  Files.move(temp, target,
    StandardCopyOption.REPLACE_EXISTING);
33                  System.out.println("Atomic move не поддерживается,
    выполнена обычная замена");
34              }
35
36              System.out.println(Files.readString(target,
    StandardCharsets.UTF_8));
37          } catch (IOException e) {
38              System.err.println("Ошибка: " + e.getMessage());
39              System.exit(1);
40          } finally {
41              if (temp != null) {
42                  try {
43                      Files.deleteIfExists(temp);
44                  } catch (IOException ignored) {
45                  }
46              }
47          }
48      }
49  }
```

Листинг – 16 7/Main.java

```
1   /*
2   Задание 7. Частично повреждённый импорт
3
4   Импортёр собирает ошибки с номерами строк и не публикует половину
    каталога. Выберите atomic all-or-nothing либо partial result и
    зафиксируйте контракт типом результата.
5   */
6   import java.io.IOException;
```

```
7     import java.nio.charset.StandardCharsets;
8     import java.nio.file.Files;
9     import java.nio.file.Path;
10    import java.util.ArrayList;
11    import java.util.List;
12
13    public class Main {
14        record Result(List<String> courses, List<String> errors) {
15        }
16
17        static Result importFile(Path path) throws IOException {
18            List<String> courses = new ArrayList<>();
19            List<String> errors = new ArrayList<>();
20            List<String> lines = Files.readAllLines(path,
      StandardCharsets.UTF_8);
21
22            for (int i = 0; i < lines.size(); i++) {
23                String[] parts = lines.get(i).split(";");
24                if (parts.length != 2 || parts[0].isBlank()) {
25                    errors.add("строка " + (i + 1) + ": нужно
      имя;часы");
26                    continue;
27                }
28                try {
29                    int hours = Integer.parseInt(parts[1]);
30                    courses.add(parts[0] + " — " + hours);
31                } catch (NumberFormatException e) {
32                    errors.add("строка " + (i + 1) + ": неверные часы");
33                }
34            }
35
36            if (!errors.isEmpty()) {
37                return new Result(List.of(), errors);
38            }
39            return new Result(courses, List.of());
40        }
41
42        public static void main(String[] args) throws IOException {
43            Path path = Files.createTempFile("import", ".csv");
44            Files.writeString(path, "Java;72\nСети;много\n;10\n",
      StandardCharsets.UTF_8);
45
46            Result result = importFile(path);
47            if (!result.errors().isEmpty()) {
48                System.out.println("Импорт отклонён целиком:");
49                result.errors().forEach(System.out::println);
```

```
50            } else {
51                System.out.println("Опубликовано: " + result.courses());
52            }
53
54            Files.deleteIfExists(path);
55        }
56    }
```

Листинг – 17 8/Main.java

```
1     /*
2     Задание 8. Потоковый импорт большого файла
3
4     Обработайте файл, превосходящий доступную память, без
      `readAllLines`. Поддержите счётчики, ограниченный список ошибок и
      отмену после заданного порога.
5     */
6     import java.io.BufferedReader;
7     import java.io.BufferedWriter;
8     import java.io.IOException;
9     import java.nio.charset.StandardCharsets;
10    import java.nio.file.Files;
11    import java.nio.file.Path;
12    import java.util.ArrayList;
13    import java.util.List;
14
15    public class Main {
16        public static void main(String[] args) throws IOException {
17            Path path = Files.createTempFile("big", ".csv");
18
19            try (BufferedWriter writer = Files.newBufferedWriter(path,
      StandardCharsets.UTF_8)) {
20                for (int i = 1; i <= 1000; i++) {
21                    writer.write("Курс " + i + ";" + (i % 100 == 0 ?
      "ошибка" : "72"));
22                    writer.newLine();
23                }
24            }
25
26            int lines = 0;
27            int good = 0;
28            int errors = 0;
29            List<String> firstErrors = new ArrayList<>();
30
31            try (BufferedReader reader = Files.newBufferedReader(path,
      StandardCharsets.UTF_8)) {
32                String line;
```

```
33                while ((line = reader.readLine()) != null) {
34                    lines++;
35                    String[] parts = line.split(";");
36                    try {
37                        Integer.parseInt(parts[1]);
38                        good++;
39                    } catch (RuntimeException e) {
40                        errors++;
41                        if (firstErrors.size() < 5) {
42                            firstErrors.add("строка " + lines);
43                        }
44                    }
45                    if (errors >= 10) {
46                        System.out.println("Отмена после 10 ошибок");
47                        break;
48                    }
49                }
50            }
51
52            System.out.println("Строк: " + lines + ", корректных: " +
      good + ", ошибок: " + errors);
53            System.out.println("Первые ошибки: " + firstErrors);
54            Files.deleteIfExists(path);
55        }
56    }
```

Листинг – 18 9/Main.java

```
1     /*
2     Задание 9. Версионируемый формат
3
4     Добавьте schema version, checksum и migration из версии 1 в 2.
      Неизвестная будущая версия не читается как старая. Повреждение
      обнаруживается до публикации объектов.
5     */
6     import java.io.ByteArrayInputStream;
7     import java.io.ByteArrayOutputStream;
8     import java.io.DataInputStream;
9     import java.io.DataOutputStream;
10    import java.io.IOException;
11    import java.nio.file.Files;
12    import java.nio.file.Path;
13    import java.util.ArrayList;
14    import java.util.List;
15    import java.util.zip.CRC32;
16
17    public class Main {
18        static final int MAGIC = 0xCAFE;
```

```
19
20        record Course(String name, int hours, int credits) {
21        }
22
23        static byte[] payload(int version, List<Course> courses) throws
    IOException {
24            ByteArrayOutputStream bytes = new ByteArrayOutputStream();
25            try (DataOutputStream out = new DataOutputStream(bytes)) {
26                out.writeInt(courses.size());
27                for (Course course : courses) {
28                    out.writeUTF(course.name());
29                    out.writeInt(course.hours());
30                    if (version == 2) {
31                        out.writeInt(course.credits());
32                    }
33                }
34            }
35            return bytes.toByteArray();
36        }
37
38        static void write(Path path, int version, List<Course> courses)
    throws IOException {
39            byte[] data = payload(version, courses);
40            CRC32 crc = new CRC32();
41            crc.update(data);
42
43            try (DataOutputStream out = new
    DataOutputStream(Files.newOutputStream(path))) {
44                out.writeInt(MAGIC);
45                out.writeInt(version);
46                out.writeInt(data.length);
47                out.writeInt((int) crc.getValue());
48                out.write(data);
49            }
50        }
51
52        static List<Course> read(Path path) throws IOException {
53            try (DataInputStream in = new
    DataInputStream(Files.newInputStream(path))) {
54                if (in.readInt() != MAGIC) {
55                    throw new IOException("неверный magic");
56                }
57                int version = in.readInt();
58                if (version != 1 && version != 2) {
59                    throw new IOException("неизвестная версия: " +
    version);
60                }
```

```
61                int length = in.readInt();
62                int checksum = in.readInt();
63                byte[] data = in.readNBytes(length);
64
65                CRC32 crc = new CRC32();
66                crc.update(data);
67                if ((int) crc.getValue() != checksum) {
68                    throw new IOException("checksum не совпал");
69                }
70
71                try (DataInputStream payload = new DataInputStream(new
    ByteArrayInputStream(data))) {
72                    int count = payload.readInt();
73                    List<Course> courses = new ArrayList<>();
74                    for (int i = 0; i < count; i++) {
75                        String name = payload.readUTF();
76                        int hours = payload.readInt();
77                        int credits = version == 1 ? Math.max(1, hours /
    36) : payload.readInt();
78                        courses.add(new Course(name, hours, credits));
79                    }
80                    return courses;
81                }
82            }
83        }
84
85        public static void main(String[] args) throws IOException {
86            Path path = Files.createTempFile("catalog", ".bin");
87            List<Course> courses = List.of(new Course("Java", 72, 2));
88
89            write(path, 1, courses);
90            System.out.println("v1 -> v2: " + read(path));
91
92            write(path, 2, courses);
93            System.out.println("v2: " + read(path));
94
95            byte[] bad = Files.readAllBytes(path);
96            bad[bad.length - 1] ^= 1;
97            Files.write(path, bad);
98            try {
99                read(path);
100           } catch (IOException e) {
101               System.out.println("Повреждение: " + e.getMessage());
102           }
103
104           Files.deleteIfExists(path);
```

```
105         }
106     }
```